

NEO Operator AWS 배포 기록

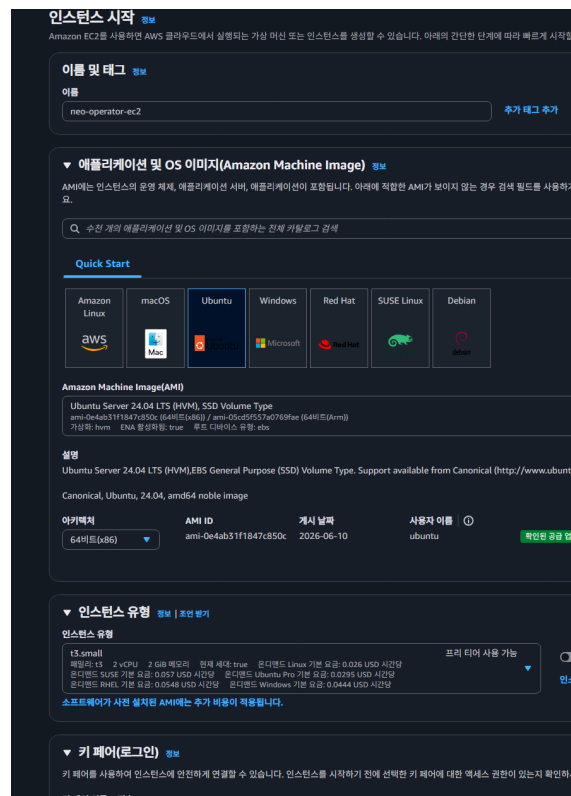
NEO Operator를 AWS EC2 환경에 배포하고, 프론트엔드와 백엔드 API, Neo4j, VectorDB, NEMI 검색 API를 Docker Compose로 실행한 과정을 정리했다.

배포 개요

- 배포 대상: NEO Operator
- 클라우드 환경: AWS EC2
- 리전: Asia Pacific (Seoul), ap-northeast-2
- 인스턴스: neo-operator-ec2
- OS: Ubuntu Server 24.04 LTS
- 인스턴스 유형: t3.small
- 고정 IP: Elastic IP 적용
- 실행 방식: Docker Compose
- 주요 구성: Vue/Nginx Frontend, FastAPI Backend, Neo4j, Qdrant, NEMI API

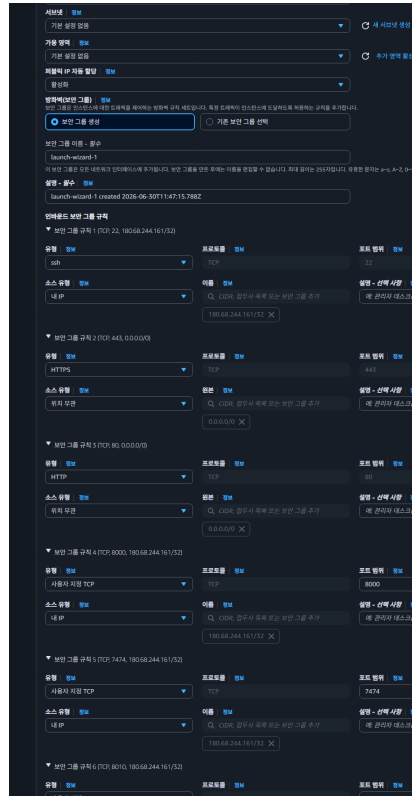
3. EC2 인스턴스 구성

EC2 인스턴스 이름은 neo-operator-ec2로 지정했다. 운영체제는 Ubuntu Server 24.04 LTS를 선택했고, 타입은 Docker 기반 멀티 컨테이너 실행을 고려해 t3.small로 구성했다.



EC2 기본 구성

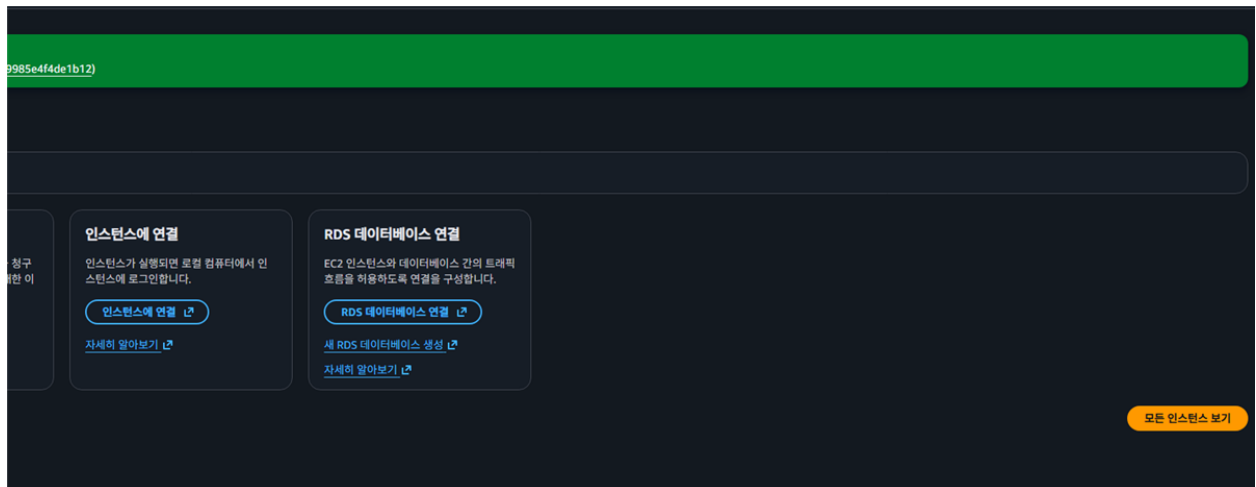
네트워크 설정에서는 SSH 접속, 웹 접속, 백엔드 확인용 포트를 분리해 보안 그룹을 구성했다. 80/443 포트는 웹 접속을 위해 열고, SSH 및 개발 확인용 포트는 제한된 접근 범위로 설정했다.



보안 그룹 및 포트 설정

4. 인스턴스 생성 및 실행 확인

설정 완료 후 인스턴스를 생성했다. 생성 직후 AWS 콘솔에서 인스턴스 시작 성공 메시지를 확인했다.



인스턴스 생성 완료

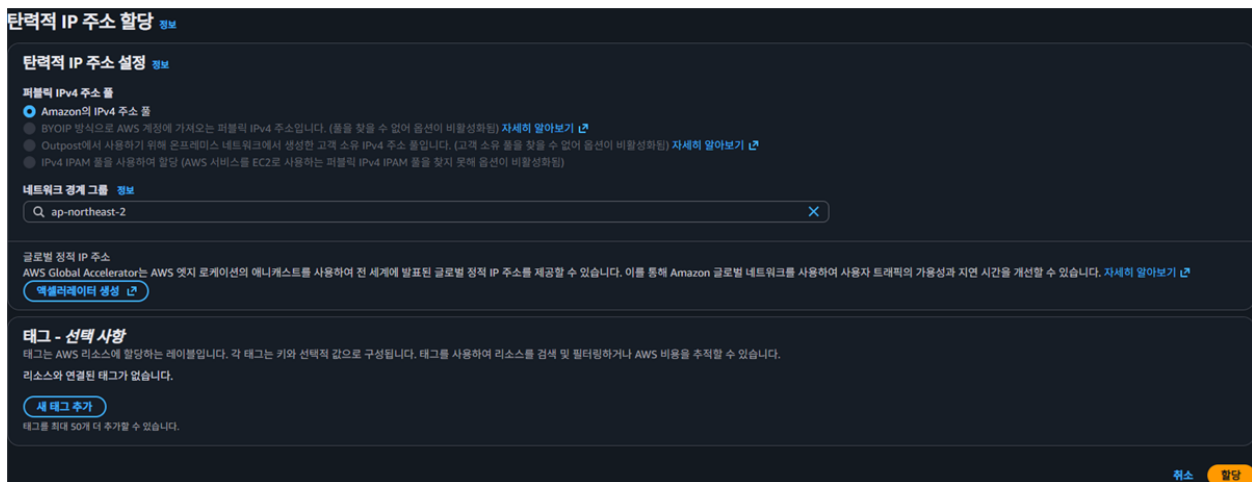
EC2 인스턴스 목록에서 neo-operator-ec2가 실행 중 상태로 표시되는 것을 확인했다.



인스턴스 실행 상태

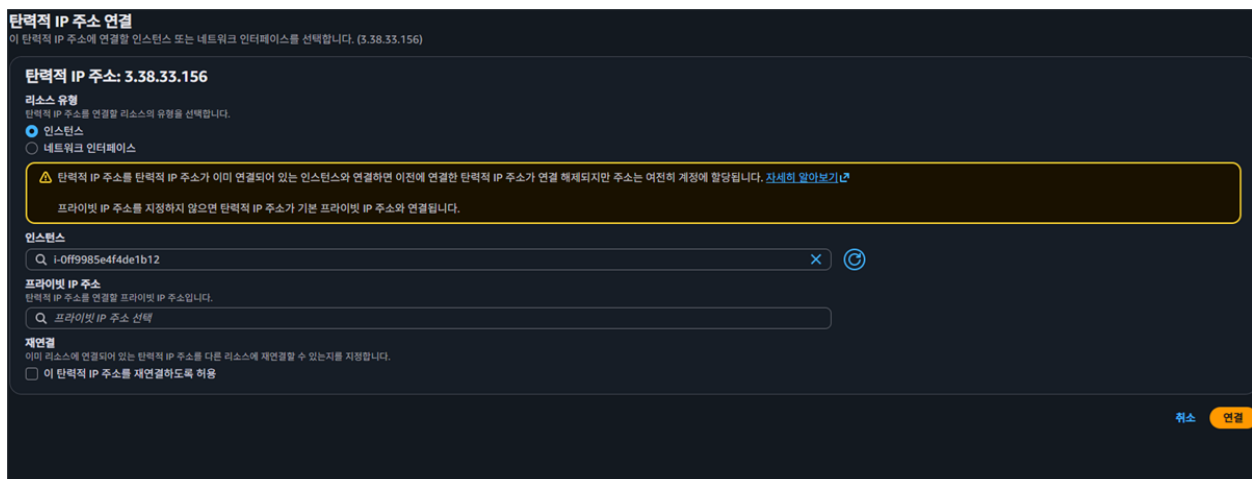
5. Elastic IP 적용

EC2의 기본 퍼블릭 IP는 인스턴스 재시작 시 변경될 수 있으므로, 고정 접속 주소를 확보하기 위해 Elastic IP를 할당했다.



Elastic IP 할당

할당한 Elastic IP를 생성한 EC2 인스턴스에 연결했다.



Elastic IP 연결 설정

연결 완료 후 Elastic IP가 인스턴스에 정상 연결된 상태를 확인했다. 계정 식별 정보는 문서 공유를 고려해 마스킹했다.



Elastic IP 연결 완료

6. SSH 접속 확인

로컬에 저장한 .pem 키 파일의 권한을 조정한 뒤 SSH로 EC2에 접속했다. 접속 후 Ubuntu 24.04 LTS 환경으로 정상 진입되는 것을 확인했다.

```
Successfully processed 1 files; Failed processing 0 files

D:\aws>ssh -i neo-operator-key.pem ubuntu@3.38.33.156
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jun 30 12:26:41 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   6.4% of 28.02GB Processes:    109
Memory usage: 11%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.47.239

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-47-239: ~$
```

SSH 접속 확인

7. 서버 실행 환경 구성

배포 서버에서 Docker, Docker Compose, Git, Nginx 설치 상태를 확인했다. Nginx는 이후 80번 포트 충돌을 피하기 위해 Docker 프론트엔드 컨테이너가 포트를 사용하도록 정리했다.

```

Last login: Tue Jun 30 12:26:42 2026 from 180.68.244.161
ubuntu@ip-172-31-47-239:~$ docker --version
Docker version 29.6.1, build 8900f1d
ubuntu@ip-172-31-47-239:~$ docker compose version
Docker Compose version v5.2.0
ubuntu@ip-172-31-47-239:~$ git --version
git version 2.43.0
ubuntu@ip-172-31-47-239:~$ nginx -v
nginx version: nginx/1.24.0 (Ubuntu)
ubuntu@ip-172-31-47-239:~$

```

서버 실행 환경 확인

8. 프로젝트 파일 업로드

로컬에서 준비한 NEO 프로젝트 파일을 EC2의 ~/neo-operator 디렉터리로 업로드했다. 업로드 대상은 Docker Compose 구성, 프론트엔드, 백엔드 API, KB, NEMI 관련 파일이다.

```

Last login: Tue Jun 30 12:34:00 2026 from 180.68.244.161
ubuntu@ip-172-31-47-239:~$ ls -la ~/neo-operator
total 32
drwxrwxr-x 6 ubuntu ubuntu 4096 Jun 30 12:41 .
drwxr-x--- 5 ubuntu ubuntu 4096 Jun 30 12:41 ..
drwx---rwx 3 ubuntu ubuntu 4096 Jun 30 12:41 deploy
-rw-rw-r-- 1 ubuntu ubuntu 1842 Jun 30 12:41 docker-compose.neo.yml
drwx---rwx 12 ubuntu ubuntu 4096 Jun 30 12:41 frontend
drwx---rwx 2 ubuntu ubuntu 4096 Jun 30 12:41 kb
-rw-rw-r-- 1 ubuntu ubuntu 82 Jun 30 12:41 requirements.txt
drwx---rwx 10 ubuntu ubuntu 4096 Jun 30 12:41 src
ubuntu@ip-172-31-47-239:~$

```

프로젝트 파일 업로드 확인

9. 환경 변수 설정

Docker Compose 실행에 필요한 환경 변수를 .env 파일로 정리했다. Neo4j 비밀번호와 포트 매핑, NEMI API URL을 설정했다. 비밀번호 값은 문서 공유를 고려해 마스킹했다.

```

ubuntu@ip-172-31-47-239:~/neo-operator$ nano .env
ubuntu@ip-172-31-47-239:~/neo-operator$ cat > .env <<'EOF'
> NEO4J_PASSWORD=NeoLocalPass2026!
> NEO_API_HOST_PORT=*****
> NEO_FRONTEND_HOST_PORT=80
> NEO4J_HTTP_HOST_PORT=7474
> NEO4J_BOLT_HOST_PORT=7687
> QDRANT_HOST_PORT=6333
> NEMI_API_HOST_PORT=8010
> NEO_NEMI_API_URL=http://nemi-api:8010/retrieve
> EOF
ubuntu@ip-172-31-47-239:~/neo-operator$ cat .env
NEO4J_PASSWORD=NeoLocalPass2026!
NEO_API_HOST_PORT=*****
NEO_FRONTEND_HOST_PORT=80
NEO4J_HTTP_HOST_PORT=7474
NEO4J_BOLT_HOST_PORT=7687
QDRANT_HOST_PORT=6333
NEMI_API_HOST_PORT=8010
NEO_NEMI_API_URL=http://nemi-api:8010/retrieve
ubuntu@ip-172-31-47-239:~/neo-operator$

```

환경 변수 설정

10. Docker Compose 빌드 및 실행

Docker Compose로 프론트엔드와 백엔드 구성 요소를 빌드했다. 프론트엔드는 Nginx 기반 정적 빌드로 실행되고, 백엔드는 FastAPI 컨테이너로 실행된다.

```
ubuntu@ip-172-31-47-239:~/neo-operator$ docker compose -f docker-compose.neo.yml up -d --build
[+] up 17/17
✔ Image qdrant/qdrant:v1.12.6 Pulled 11.0s
✔ Image neo4j:5-community Pulled 14.8s
[+] Building 74.9s (43/43) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.43kB 0.0s
=> [frontend internal] load build definition from frontend.Dockerfile 0.1s
=> => transferring dockerfile: 309B 0.0s
=> [neo-api internal] load build definition from neo-api.Dockerfile 0.1s
=> => transferring dockerfile: 986B 0.0s
=> [nemi-api internal] load build definition from nemi-api.Dockerfile 0.1s
=> => transferring dockerfile: 474B 0.0s
=> [frontend internal] load metadata for docker.io/library/node:22-alpine 2.0s
=> [frontend internal] load metadata for docker.io/library/nginx:1.27-alpine 1.9s
=> [neo-api internal] load metadata for docker.io/library/python:3.11-slim 1.9s
=> [neo-api internal] load metadata for docker.io/library/ubuntu:22.04 1.9s
=> [frontend internal] load .dockerignore 0.0s
```

Docker 빌드

컨테이너 실행 상태를 확인했다. 최종 구성은 프론트엔드, FastAPI, Neo4j, Qdrant, NEMI API가 함께 실행되는 구조다.

```
ubuntu@ip-172-31-47-239:~/neo-operator$ sudo systemctl stop nginx
ubuntu@ip-172-31-47-239:~/neo-operator$ sudo systemctl disable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable nginx
Removed /etc/systemd/system/multi-user.target.wants/nginx.service".
ubuntu@ip-172-31-47-239:~/neo-operator$ docker compose -f docker-compose.neo.yml up -d frontend
[+] up 5/5
✔ Container neo-operator-nemi-api-1 Running 0.0s
✔ Container neo-operator-neo-api-1 Running 0.0s
✔ Container neo-operator-neo4j-1 Healthy 0.5s
✔ Container neo-operator-vectordb-1 Running 0.0s
✔ Container neo-operator-frontend-1 Started 0.2s
ubuntu@ip-172-31-47-239:~/neo-operator$ docker compose -f docker-compose.neo.yml ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
neo-operator-frontend-1            neo-operator-frontend              "/docker-entrypoint...."          frontend   5 minutes ago  Up 3 seconds
neo-operator-nemi-api-1            neo-operator-nemi-api              "python -m uvicorn s..."          nemi-api   5 minutes ago  Up 5 minutes
neo-operator-neo-api-1            neo-operator-neo-api              "python -m uvicorn a..."          neo-api    5 minutes ago  Up 4 minutes
neo-operator-neo4j-1              neo4j:5-community                 "tini -g -- /startup..."          neo4j      5 minutes ago  Up 5 minutes (hea
lthy)
neo-operator-vectordb-1            qdrant/qdrant:v1.12.6              "./entrypoint.sh"                  vectordb   5 minutes ago  Up 5 minutes
```

컨테이너 실행 상태

11. HTTP 접속 검증

EC2 내부와 외부 주소에서 HTTP 응답을 확인했다. /neo 경로가 정상 응답을 반환하는 것을 확인했다.

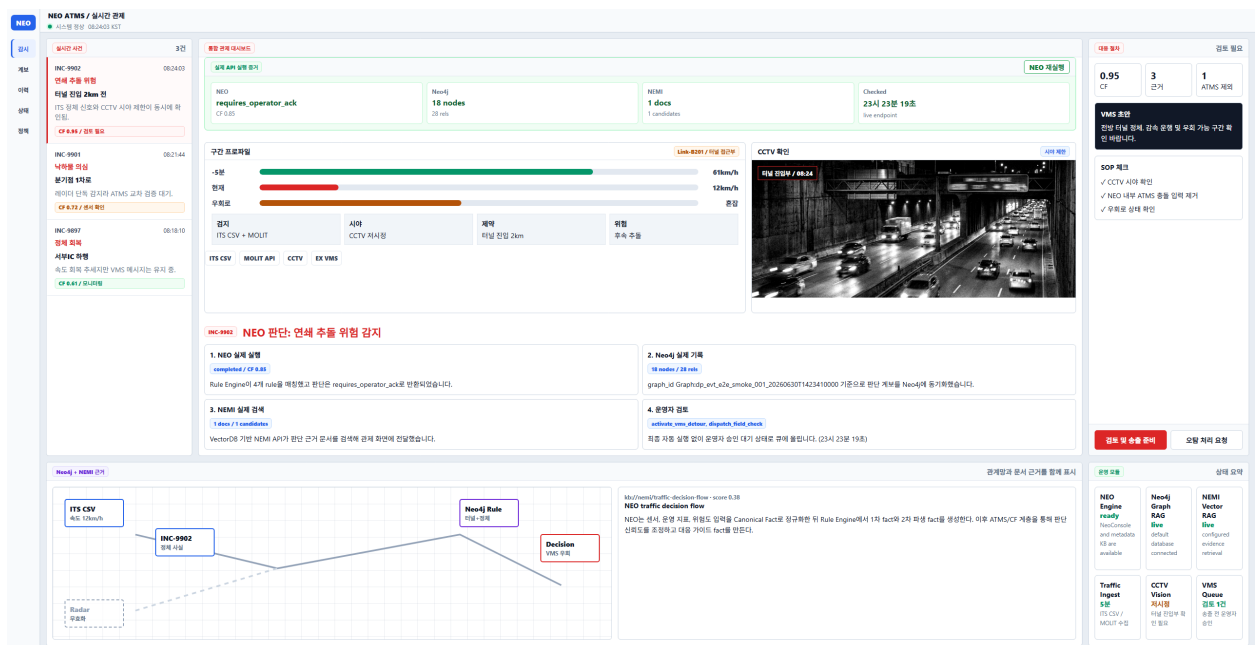
```
ubuntu@ip-172-31-47-239:~/neo-operator$ curl -I http://localhost/neo
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Tue, 30 Jun 2026 12:56:35 GMT
Content-Type: text/html
Content-Length: 558
Last-Modified: Tue, 30 Jun 2026 12:48:26 GMT
Connection: keep-alive
ETag: "6a43bb1a-22e"
Accept-Ranges: bytes

ubuntu@ip-172-31-47-239:~/neo-operator$ curl -I http://3.38.33.156/neo
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Tue, 30 Jun 2026 12:59:20 GMT
Content-Type: text/html
Content-Length: 558
Last-Modified: Tue, 30 Jun 2026 12:48:26 GMT
Connection: keep-alive
ETag: "6a43bb1a-22e"
Accept-Ranges: bytes

ubuntu@ip-172-31-47-239:~/neo-operator$
```

HTTP 응답 확인

브라우저에서 Elastic IP 기반 URL로 접속해 NEO Operator 화면이 표시되는 것을 확인했다.



NEO 웹 화면 확인

12. 배포 중 처리한 이슈

- 80번 포트 충돌: EC2에 설치된 호스트 Nginx가 80번 포트를 사용하고 있어 Docker 프론트엔드 컨테이너가 포트를 바인딩하지 못했다. 호스트 Nginx를 중지하고 비활성화한 뒤 프론트엔드 컨테이너를 재실행했다.
- 프론트엔드 경로 혼동: 최초 업로드 시 포트폴리오 프론트엔드가 배포되어 /neo 경로에 잘못된 화면이 표시됐다. NEO 전용 Vue 프론트엔드로 교체 후 재빌드했다.
- NEMI 검색 결과 0건: 일반 decision package 기반 검색에서는 문서가 반환되지 않아, NEMI corpus의 evidence type과 query key를 맞춰 검색 결과가 표시되도록 수정했다.

최종 확인

- https://3-38-33-156.sslip.io/neo 접속 확인
- /neo, /neo/lineage, /neo/logs, /neo/health, /neo/settings 라우트 정상 응답 확인
- FastAPI /api/v1/runtime/status 응답 확인
- NEO 판단 실행, Neo4j graph sync, NEMI retrieval 연결 확인

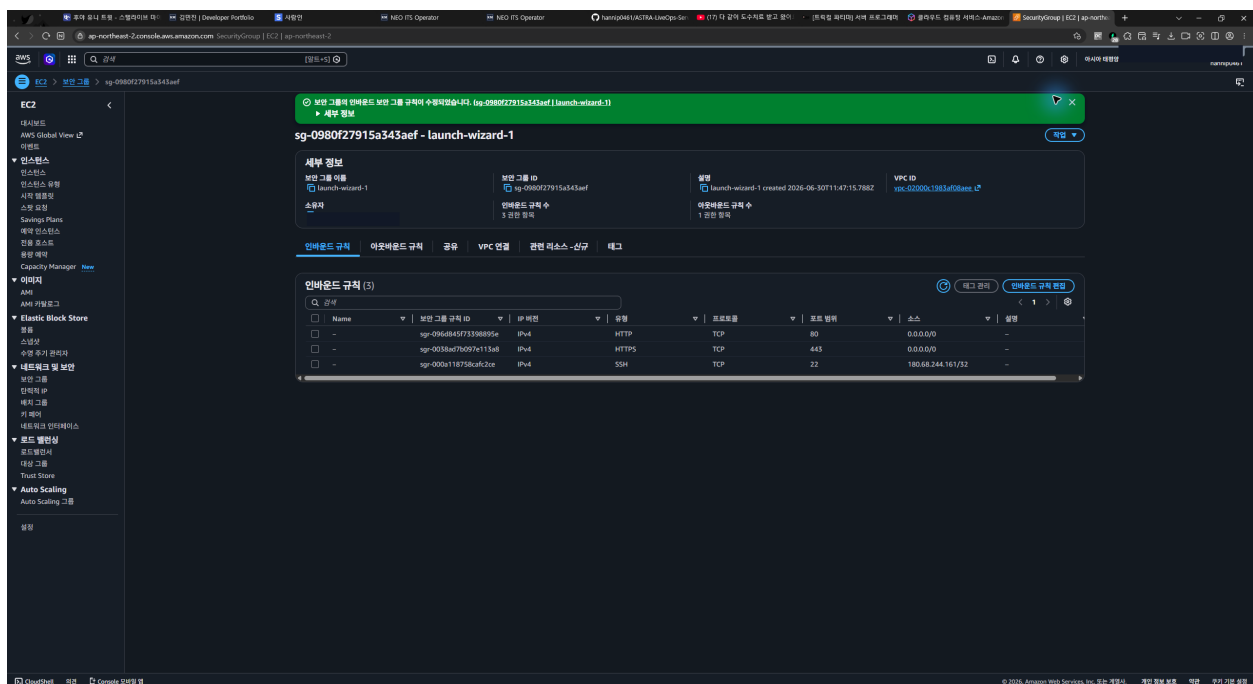
현재 배포는 AWS EC2에서 Docker Compose 기반으로 실행되며, NEO Operator의 프론트엔드와 백엔드 주요 구성 요소가 하나의 서버에서 동작하는 상태다.

13. 운영 포트 보안 점검

애플리케이션 갱신 후 Docker 포트 바인딩과 EC2 보안 그룹을 함께 점검했다. 프론트엔드는 호스트 Nginx를 통해 HTTP/HTTPS로 제공하고, FastAPI·NEMI API·Neo4j·Qdrant는 EC2의 loopback 주소(127.0.0.1)에만 바인딩했다.

- 보안 그룹 허용: HTTP 80과 HTTPS 443은 전체 접속 허용
- SSH 22: 관리 PC 공인 IP의 /32 범위만 허용
- 제거한 개발 규칙: NEMI API 8010, Neo4j Browser 7474, FastAPI 8000
- 외부 포트 재검증: 22/80/443 열림, 6333/7474/7687/8000/8010 닫힘

이를 통해 웹 화면과 제한된 SSH 접속만 허용하고, 내부 API와 데이터 저장소는 인터넷에 직접 노출되지 않도록 구성했다.



최종 보안 그룹 인바운드 규칙

14. HTTPS 적용

IP 기반 임시 도메인 3-38-33-156.sslip.io가 Elastic IP를 가리키도록 확인한 뒤, 프론트엔드 컨테이너를 127.0.0.1:8080에만 바인딩했다. 호스트 Nginx는 외부의 80/443번 요청을 받아 프론트엔드 컨테이너로 전달하도록 구성했다.

Certbot과 Let's Encrypt를 이용해 TLS 인증서를 발급하고 HTTP 요청을 HTTPS로 자동 전환했다. 인증서 자동 갱신 모의시험도 정상 통과했다.

- 최종 URL: <https://3-38-33-156.sslip.io/neo>
- HTTP 요청: HTTPS로 301 전환
- 인증서 만료일: 2026-10-11
- 자동 갱신: Certbot 예약 작업 활성화 및 `renew --dry-run` 성공
- 전체 Vue 라우트 및 FastAPI 프록시 응답 확인

The screenshot displays the NEO Operator interface, which is a dashboard for managing traffic incidents and road conditions. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for '운영' (Operation), '감시' (Monitoring), '이력' (History), '분석' (Analysis), '개보' (Maintenance), '관리' (Management), '상태' (Status), and '정책' (Policy).
- Top Header:** Shows 'NEO ATMS / 실시간 관제' (NEO ATMS / Real-time Monitoring) and '시스템 정상 08:24:03 KST 사전 선택과 조치 검토' (System Normal 08:24:03 KST Pre-selection and Action Review).
- Main Content Area:**
 - 실시간 사건 (Real-time Incident):** Lists incidents such as 'INC-9902' (연쇄 주돌 위험 - Chain collision risk) and 'INC-9901' (낙하물 의심 - Falling object suspected).
 - 통합 관제 대시보드 (Integrated Monitoring Dashboard):** Provides a comprehensive overview of road conditions, including '운행자 검토 단계' (Driver review stages) and '실시간 판단 확인' (Real-time decision confirmation).
 - 구간 프로파일 (Section Profile):** Displays speed and traffic data for specific road sections, such as 'INC-9902'.
 - CCTV 확인 (CCTV Confirmation):** Shows live camera footage of the road.
- Right Sidebar:** Contains '대응 절차' (Response Procedure) and 'VMS 조안' (VMS Confirmation) sections, detailing the steps for handling incidents and confirming VMS status.

HTTPS로 배포된 NEO Operator